

Semana 1 – Guía de prácticas (2h)

Objetivo: Dejar operativo el entorno (VS Code + Node.js) y construir una **página base HTML5 semántica y accesible** que sirva de esqueleto para los retos posteriores.

RDAs asociados: RDA 1 (fundamentos para diseño e implementación correcta).

Requisitos previos

- **SO:** Windows/macOS/Linux
- **Software:** Navegador moderno (Chrome/Edge/Firefox), conexión a internet (solo para instalación).
- **Conocimientos:** Estructura básica de archivos, navegación por carpetas, nociones introductorias de HTML.

Bloque 0 (20 min) — Principiantes: HTML, CSS y DOM + “Hola Mundo”

0.1 ¿Qué es HTML, CSS y el DOM? (5 min)

- **HTML:** lenguaje de **marcado** → define la **estructura** y significado del contenido (títulos, párrafos, listas, imágenes, enlaces).
- **CSS:** lenguaje de **estilos** → define la **apariencia** (colores, tamaños, disposición).
- **DOM** (*Document Object Model*): representación en **árbol** del documento en memoria cuando el navegador carga el HTML. Cada etiqueta es un *nodo* (padres, hijos, hermanos).
 - **Idea clave:** el HTML es el “plano”; el DOM es el “árbol vivo” que el navegador puede modificar.

0.2 Tu primer “Hola Mundo” (8 min)

1. Crea una carpeta temporal: hola-mundo/.
2. Dentro, crea index.html con lo mínimo:

```
<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <title>Hola Mundo</title>
  <meta name="viewport" content="width=device-width, initial-scale=1">
</head>
<body>
  <h1>¡Hola Mundo!</h1>
  <p>Mi primera página web.</p>
</body>
</html>
```

3. Abre el archivo en el navegador (doble clic o arrástralo al navegador).
4. (Opcional) Crea styles.css:
- 5.

```
body { font-family: system-ui, sans-serif; line-height: 1.5; margin: 2rem; }
h1 { font-size: 2rem; }
```

y **enlázalo** en el <head>:

<link rel="stylesheet" href="styles.css">

0.3 Ver el DOM y tocarlo (7 min)

- Abre **DevTools** (Chrome/Edge: Ctrl+Shift+I, macOS: Cmd+Opt+I).
- Pestaña **Elements**: verás la estructura **DOM**.
- Haz doble clic sobre <h1> y cambia el texto → observa que el **DOM cambia en vivo**, aunque el archivo **HTML no se modifica** (si recargas, vuelve al original).
- Diferencia **Ver código fuente** (Ctrl+U) vs **Elements**: el primero muestra el HTML de archivo; el segundo, el DOM actual.

0.4 Micro-ejercicios (no calificados, 0–5 min)

- Agrega un **enlace**: Sitio PUCE (texto del enlace debe describir el destino).
- Agrega una **lista** con 3 ítems.
- Agrega una **imagen** con alt **descriptivo** (si no tienes imagen, déjala para luego).

Hora 1 (≈60 min) – Entorno listo + proyecto base

1) Verifica el entorno (5 min)

- Abre una terminal y ejecuta:
 - node -v → debería devolver una versión (LTS recomendada).
 - npm -v → debería devolver la versión de npm.
- Si alguno falla, procede a instalar.

2) Instala Node.js LTS (10–15 min)

- Descarga e instala **Node.js (LTS)** para tu sistema operativo.
- Cierra y reabre la terminal. Repite node -v y npm -v.

3) Instala VS Code + extensiones clave (10 min)

- Instala **Visual Studio Code**.
- Abre VS Code y agrega extensiones:
 - *Live Server* (vista previa local de HTML).
 - *Prettier* (formateo).
 - *ESLint* (lo usaremos más adelante).
 - *HTMLHint* (opcional, validación HTML).

4) Crea la estructura del proyecto (10 min)

- Crea una carpeta de trabajo, por ejemplo: semana1-plataformas/.
- Dentro, crea:
 - semana1-plataformas/
 - └─ public/

- | └─ index.html
- | └─ assets/
- | └─ images/ (vacío por ahora)
- | └─ styles/
- | └─ styles.css
- └─ README.md
- Abre la carpeta en VS Code (**File > Open Folder**).

5) Inicializa Git + npm (10 min)

1. Abre **PowerShell** como usuario normal y ejecuta:

```
winget install --id Git.Git -e
```

Si te pide confirmar, acepta.

2. Cierra **todas** las terminales/VS Code y vuelve a abrir.
3. Verifica:

```
git --version
```

- En la terminal integrada (View > Terminal):
 - git init
 - npm init -y (crea package.json)
- crea .gitignore
 - En VS Code, menú **File → New File**.
 - Escribe el nombre **.gitignore** (con el punto primero) y guarda en la raíz del proyecto.
 - Pega esto y guarda:

```
node_modules/
.env
.DS_Store
```

node_modules/ ignora la carpeta completa, .env ignora el archivo de variables, .DS_Store ignora el archivo de macOS.

6) Crea el esqueleto HTML semántico (10 min)

En public/index.html, pega y **ajusta** (metadatos, título, idioma):

```
<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8" />
  <title>Base – Sistema de Gestión de Procesos</title>
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <meta name="description" content="Esqueleto semántico y accesible para el curso
  Desarrollo en Plataformas." />
  <link rel="stylesheet" href="../styles/styles.css" />
</head>
```

```

<body>
  <a class="skip-link" href="#contenido">Saltar al contenido</a>

  <header>
    <h1>Desarrollo en Plataformas</h1>
    <nav aria-label="Navegación principal">
      <ul>
        <li><a href="#" aria-current="page">Inicio</a></li>
        <li><a href="#modulos">Módulos</a></li>
        <li><a href="#accesibilidad">Accesibilidad</a></li>
      </ul>
    </nav>
  </header>

  <main id="contenido">
    <section aria-labelledby="sec-intro">
      <h2 id="sec-intro">Introducción</h2>
      <p>Este es el esqueleto base para el proyecto del curso. Prioriza semántica y
accesibilidad.</p>
    </section>

    <section id="modulos" aria-labelledby="sec-modulos">
      <h2 id="sec-modulos">Módulos</h2>
      <article aria-labelledby="art-turnos">
        <h3 id="art-turnos">Módulo: Gestión de Turnos</h3>
        <p>Contenido placeholder. Aquí, más adelante, se integrará cliente-servidor.</p>
      </article>
      <aside aria-label="Información complementaria">
        <p>Nota: Mantén una jerarquía de encabezados coherente (h1→h2→h3).</p>
      </aside>
    </section>

    <section id="accesibilidad" aria-labelledby="sec-a11y">
      <h2 id="sec-a11y">Accesibilidad</h2>
      <p role="note" aria-live="polite">Recuerda: texto alternativo en imágenes y formularios
etiquetados.</p>
    </section>
  </main>

  <footer>
    <p>&copy; 2025. Curso Desarrollo en Plataformas.</p>
  </footer>
</body>
</html>

```

7) Estilos mínimos + foco visible (5 min)

En styles/styles.css:

```

* { box-sizing: border-box; }
body { font-family: system-ui, sans-serif; line-height: 1.5; margin: 0; }

```

```

header, main, footer { padding: 1rem; }
.skip-link {
  position: absolute; left: -999px; top: -999px;
}
.skip-link:focus {
  position: static; background: #eee; padding: .5rem;
}
nav ul { list-style: none; padding: 0; display: flex; gap: 1rem; }
a:focus { outline: 3px solid #000; }

```

8) Vista previa local (5 min)

- Abre public/index.html y ejecuta **Open with Live Server**.
- Verifica que el *skip link* funcione con **Tab** (debe aparecer y llevar al <main>).

Criterios de logro de la Hora 1:

- Node/npm y VS Code instalados.
- Proyecto con estructura básica y Git iniciado.
- Página HTML semántica mínima visible en Live Server con foco accesible.

Hora 2 (≈60 min) – Semántica + Accesibilidad aplicada

1) Añade una tabla accesible (10 min)

Debajo de “Gestión de Turnos”, inserta:

```

<table>
  <caption>Estados del sistema por entorno</caption>
  <thead>
    <tr>
      <th scope="col">Entorno</th>
      <th scope="col">Estado</th>
      <th scope="col">Última revisión</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">Desarrollo</th>
      <td>Operativo</td>
      <td>2025-09-22</td>
    </tr>
    <tr>
      <th scope="row">Producción</th>
      <td>Operativo parcial</td>
      <td>2025-09-21</td>
    </tr>
  </tbody>
</table>

```

Claves: caption describe la tabla; th scope identifica encabezados.

2) Inserta una imagen con alt adecuado (5 min)

- Coloca una imagen (placeholder) en assets/images/.
- En el <article>:

```
<figure>
  
  <figcaption>Arquitectura cliente-servidor base del curso.</figcaption>
</figure>
```

3) Crea un formulario accesible (15 min)

Bajo la sección Accesibilidad:

```
<form aria-labelledby="tituloregistro">
  <h3 id="tituloregistro">Registro de participación</h3>

  <label for="nombre">Nombre completo</label>
  <input id="nombre" name="nombre" type="text" required aria-describedby="ayuda-
nombre">

  <p id="ayuda-nombre">Usa tu nombre institucional.</p>

  <label for="email">Correo</label>
  <input id="email" name="email" type="email" required aria-describedby="ayuda-email">
  <p id="ayuda-email">Formato: usuario@ejemplo.edu.ec</p>

  <fieldset>
    <legend>Preferencia de contacto</legend>
    <label><input type="radio" name="contacto" value="correo" checked> Correo</label>
    <label><input type="radio" name="contacto" value="telefono"> Teléfono</label>
  </fieldset>

  <button type="submit">Guardar</button>
</form>
```

Claves: cada input tiene label; *hints* con aria-describedby; fieldset/legend agrupan opciones.

4) Revisa jerarquía y navegación por teclado (10 min)

- Un solo <h1>; subsecciones con <h2> y contenidos con <h3>.
- Recorre con **Tab**: debe ser lógico (skip-link → nav → contenido → formulario → footer).
- Asegura foco visible (no lo elimines con CSS).

5) Validación rápida (10–15 min)

- **HTMLHint**: revisa advertencias; corrige etiquetas sin cerrar o jerarquía rota.
- **Lighthouse** (en DevTools): ejecuta auditoría de **Accessibility** (modo Desktop).
 - **Objetivo:** ≥ 90 en accesibilidad.

- Corrige: contraste, alt faltante, aria-label necesario, orden de encabezados.

Criterios de logro de la Hora 2:

- Tabla con caption y scope correctos.
- Imagen con alt significativo y figcaption.
- Formulario etiquetado, con *hints* y agrupaciones correctas.
- Auditoría inicial pasada y mejoras aplicadas.

Checklist de validación (rápido)

- | |
|--|
| <ul style="list-style-type: none">• ☐ Un único <h1> y jerarquía consistente.• ☐ nav con aria-label y uso de listas.• ☐ <i>Skip link</i> visible al enfocar.• ☐ Imágenes con alt apropiado (o vacío si decorativas).• ☐ Tablas con caption y scope.• ☐ Formularios con label/aria-describedby.• ☐ Foco siempre visible.• ☐ Lighthouse A11y ≥ 90. |
|--|

Errores comunes a evitar

- Usar <div> para botones/enlaces (usa <button>/<a>).
- Quitar el *outline* de foco sin alternativa visible.
- Encabezados salteados (pasar de h1 a h4).
- Texto en imágenes sin *overlay* que garantice contraste.
- alt genéricos ("imagen1") o irrelevantes.

Ejercicios para resolver en clase

Ejercicio 1 — Refactor semántico

Te entregarán un HTML "anti-patrón" (solo div/span).

Tarea: Reescríbelo semánticamente: introduce header, nav, main, section, article, aside, footer; corrige encabezados, agrega *skip link*, imagen con alt y una tabla con caption+scope.

Criterios de finalización:

- Estructura semántica completa.
- Navegación por teclado sin trampas.
- Lighthouse A11y ≥ 90 .

Ejercicio 2 — Mini landing accesible

Crea una página con:

1. **Navegación** (3 enlaces, aria-current en la sección activa).
2. **Sección principal** con un **artículo** y una **imagen** (figure+figcaption).
3. **Tabla** con caption y cabeceras correctas.

4. **Formulario** con label/aria-describedby y agrupación (fieldset).

Criterios de finalización:

- Semántica correcta; foco visible; orden lógico de Tab.
- Contraste suficiente (usa un verificador si es necesario).
- Lighthouse A11y ≥ 90 .

Hora 1 — Entorno listo + proyecto base

1) Verifica el entorno (node -v / npm -v)

- **¿Para qué?** Comprobar que tienes el *runtime* (Node.js) y el gestor de paquetes (npm) disponibles. Si no existen o están rotos, cualquier intento de automatizar, probar o desplegar fallará.
- **Competencia:** Diagnóstico temprano del entorno y reproducibilidad (base para CI/CD y trabajo en equipo).

2) Instala Node.js LTS

- **¿Para qué?** La versión **LTS** garantiza estabilidad y soporte prolongado. Evita incompatibilidades de librerías que suelen aparecer en versiones *Current*.
- **Competencia:** Elección informada de versiones para asegurar compatibilidad de dependencias (práctica profesional).

3) Instala VS Code + extensiones

- **Live Server:** **¿Para qué?** Levanta un servidor local con recarga automática. Acelera el ciclo *editar-ver*.
- **Prettier:** **¿Para qué?** Formatea el código de forma consistente; reduce discusiones de estilo y errores por sangrías.
- **ESLint (para más adelante):** **¿Para qué?** Detecta *smells* y errores lógicos en JS. Previene bugs en fases tempranas.
- **HTMLHint:** **¿Para qué?** Valida la estructura HTML (etiquetas faltantes, jerarquías mal puestas).
- **Competencia:** Calidad de código y productividad (fundamental para equipos y escalabilidad).

4) Estructura del proyecto (public/, assets/, styles/, README.md)

- **¿Para qué?** Separar responsabilidades:
 - public/ para lo que se sirve al navegador,
 - assets/ para recursos estáticos (imágenes),
 - styles/ para el CSS,
 - README.md para documentar cómo correr/instalar.
- **Competencia:** Organización profesional del repo (facilita mantenimiento, pruebas y despliegue).

5) Inicializa Git + npm

- **git init:** **¿Para qué?** Versionar el proyecto (historial, ramas, *pull requests*).
- **npm init -y:** **¿Para qué?** Crea package.json (metadatos, *scripts*, dependencias).
- **.gitignore:** **¿Para qué?** Evitar subir archivos generados o sensibles:
 - node_modules/ (pesado y regenerable),
 - .env (secretos),
 - .DS_Store (basura de macOS).
- **Competencia:** Flujo de trabajo profesional (control de versiones + gestión de dependencias).

6) Esqueleto HTML semántico

- **¿Para qué?** Establecer una **estructura correcta** desde el día 1:
 - header/nav/main/section/article/aside/footer → semántica y SEO.
 - **Skip link** → accesibilidad por teclado.
 - aria-current="page" → indica dónde está el usuario.
 - aria-live="polite" → anuncia cambios dinámicos sin interrumpir al lector de pantalla.
- **Competencia:** Base sólida para accesibilidad (WCAG) y para integrar más adelante lógica cliente-servidor.

7) Estilos mínimos + foco visible

- **¿Para qué?** Asegurar que el **foco** sea perceptible (no romper accesibilidad). El *skip link* oculto hasta foco es un patrón estándar.
- **Competencia:** Buenas prácticas de UI accesible (requisito frecuente en proyectos reales).

8) Vista previa local (Live Server)

- **¿Para qué?** Validar rápidamente que la página carga y que la navegación por teclado funciona; detectar errores de ruta y *cache busting*.
- **Competencia:** Ciclo de feedback corto (iteración ágil).

Hora 2 — Semántica + Accesibilidad aplicada

1) Tabla accesible (caption + scope)

- **¿Para qué?** Presentar datos tabulares con **relaciones claras** entre celdas y encabezados:
 - caption = propósito de la tabla,
 - scope="col/row" = asociación para lectores de pantalla.
- **Competencia:** Comunicación de datos comprensible para todos los usuarios (base para reportes y paneles futuros).

2) Imagen con alt + figure/figcaption

- **¿Para qué?**
 - alt describe la **información** de la imagen (no su estética).
 - figcaption añade contexto.
 - Si la imagen es decorativa, alt="" para **no** distraer al lector de pantalla.
- **Competencia:** Inclusión de contenidos visuales sin barreras (norma WCAG).

3) Formulario accesible (label, aria-describedby, fieldset/legend)

- **¿Para qué?**
 - label for vincula controles con su texto.
 - aria-describedby enlaza *hints* y errores al control (lectura coherente).
 - fieldset/legend agrupa opciones relacionadas (como *radios*).
- **Competencia:** Formularios usables por teclado y tecnología asistiva (crítico en apps reales: login, registro, checkout).

4) Jerarquía y navegación por teclado

- **¿Para qué?** Garantizar flujo lógico: **un solo h1**, progresión h2→h3, y orden de *Tab* predecible.
- **Competencia:** Coherencia estructural que reduce confusión y errores de accesibilidad.

5) Validación (HTMLHint + Lighthouse)

- **¿Para qué?**
 - **HTMLHint:** detección de problemas estructurales (etiquetas mal anidadas, atributos faltantes).
 - **Lighthouse:** puntaje y recomendaciones de **Accesibilidad** (contraste, *alt*, roles/labels).
- **Competencia:** Cultura de **calidad** basada en herramientas (medir, corregir, volver a medir).

Checklist de validación (¿para qué sirve?)

- **Propósito:** Lista de “salida de calidad” antes de dar por terminada la práctica. Minimiza retrabajo y asegura estándares mínimos (p. ej., Lighthouse ≥ 90 en A11y).

Errores comunes (¿por qué son problemáticos?)

- **<div> como botón/enlace:** rompe semántica, atajos y lectores de pantalla no entienden su rol.
- **Quitar el *outline* de foco:** deja “ciegos” a usuarios de teclado.
- **Saltar niveles de encabezado:** desordena la estructura y confunde a tecnologías asistivas.
- **Texto sobre imágenes sin contraste:** ilegible para varios usuarios y penalizado en auditorías.
- **alt genéricos:** no aportan información; lectores de pantalla leen algo inútil.

Ejercicios (¿qué competencia desarrolla cada uno?)

Ejercicio 1 — Refactor semántico

- **¿Para qué?** Convertir un HTML “plano” en uno **correctamente estructurado** y accesible.
- **Competencia:** Diagnóstico y mejora de código heredado (caso real en empresas).
- **Relación RDA/retos:** cimenta el RDA 1 (fundamentos) y prepara al estudiante para integrar estructuras y UI más adelante.

Ejercicio 2 — Mini landing accesible

- **¿Para qué?** Ensayar un *layout* completo con navegación, contenido, datos y formulario, **todo accesible**.
- **Competencia:** Composición integral de una página lista para evolucionar hacia cliente-servidor (semana 4+).
- **Relación RDA/retos:** provee el esqueleto sobre el cual, en semanas posteriores, se integrarán lógicas (colas, pilas, listas) y seguridad.

